

Part 1:

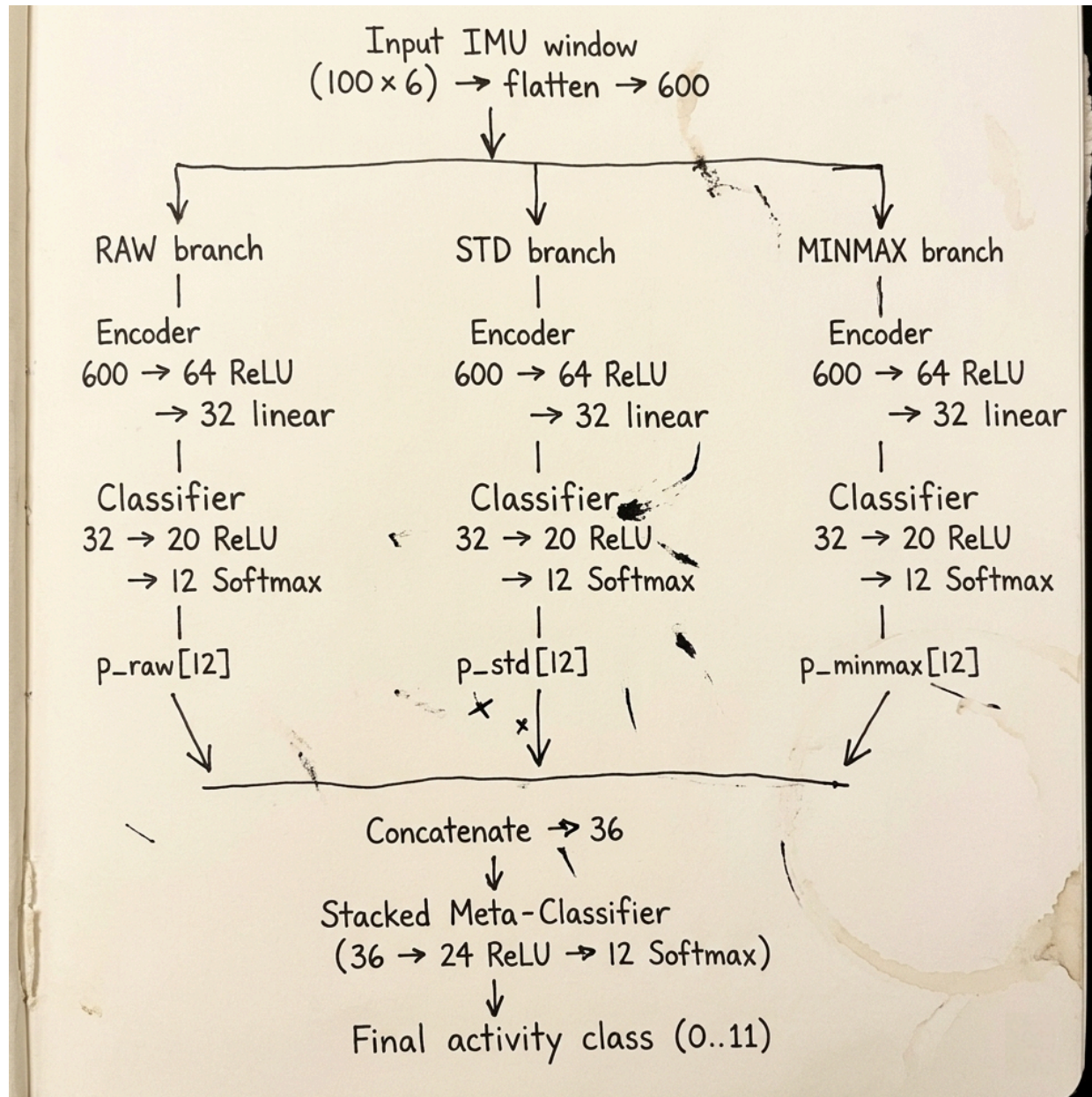
Link to Edge Impulse Project:

<https://studio.edgeimpulse.com/public/1013738/live>

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 103 ms, inference 570 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 103 ms, inference 572 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
```

Part 2:

Question 1:



Question 2:

The notebook compresses models using a pruning-to-quantization pipeline for TinyML deployment. First, pruning is applied with a polynomial sparsity schedule that increases sparsity from 10% to 80%, so more weights are gradually set to zero during training.

After pruning, the pruning wrappers are stripped to recover a normal Keras model with sparse weights, which is cleaner for the next quantization-aware training (QAT) step. Then QAT is applied to fine-tune the model while simulating low-precision quantization effects, so the model learns to keep good accuracy after integer conversion. The notebook also enforces pruning masks during and after QAT, because without this step some zeroed weights can become nonzero again, reducing the intended sparsity. Finally, the model is converted to fully int8 TFLite using representative data, which is needed to calibrate quantization scales and zero-points from real input distributions. Pruning and int8 quantization are both useful for Arduino Nano 33 BLE Sense because they reduce model size, memory use, and compute cost, making on-device inference more practical.

Question 3: Arduino Deployment and Real-World Behavior

Serial Monitor Output

```
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.8359, 0.0352, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.1289, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.7500, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.2461
Min-max-scaled model scores: 0.0000, 0.0000, 0.2852, 0.0000, 0.3828, 0.0625, 0.0000, 0.0391, 0.0000, 0.0000, 0.0000, 0.2305
Meta-classifier scores: 0.0078, 0.0352, 0.5937, 0.0312, 0.2148, 0.0195, 0.0234, 0.0156, 0.0078, 0.0352, 0.0078, 0.0078
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.5937
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0430, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9570, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.4648, 0.0000, 0.4141, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1211
Min-max-scaled model scores: 0.0000, 0.0000, 0.0547, 0.0000, 0.9414, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0000, 0.0039, 0.0586, 0.1758, 0.5352, 0.0039, 0.0078, 0.1094, 0.0000, 0.0508, 0.0430, 0.0117
-----
Final prediction: Climbing stairs
Class index: 4
Confidence score: 0.5352
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9961, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.5586, 0.4375, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.5195, 0.0000, 0.4727, 0.0000, 0.0000, 0.0078, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0000, 0.0000, 0.0664, 0.6797, 0.0039, 0.0039, 0.0000, 0.0039, 0.0000, 0.0352, 0.2109, 0.0000
-----
Final prediction: Walking
Class index: 3
Confidence score: 0.6797
-----
```

Is the output acceptable for a resting board?

Partially. The model does demonstrate meaningful sensitivity to physical orientation and motion. When the board was flat, it correctly predicted "Lying down," which is the most reasonable label for that condition. When held perpendicular it shifted toward "Walking," and when moved it responded with motion-related labels like "Running." This shows the underlying IMU signal is being interpreted in a physically meaningful way. However, predictions were not fully stable, with Window 2 producing "Climbing stairs" on a stationary flat board, indicating the model is not robust enough for consistent real-world deployment.

Observed Issues

The primary issue is confusion between "Walking" and "Running." These two classes share similar motion patterns (periodic, rhythmic acceleration along similar axes), and the model frequently misclassifies one as the other. This is visible in the base model scores: in Window 2 the raw model fires strongly on "Running" (0.957) while the other models favor "Climbing stairs" or "Lying down," and the meta-classifier struggles to reconcile this disagreement. The confidence scores across all windows are moderate (0.53 to 0.68), reflecting genuine ambiguity rather than confident correct predictions.

A secondary issue is axis orientation mismatch. The mHealth dataset was collected from sensors worn on the body at specific placements (chest, wrist, ankle), so the gravity vector and motion axes during training activities like walking and running were body-relative. On the Arduino, the axis orientation depends entirely on how the board is physically positioned, which may not align with the training setup. This explains why holding the board perpendicular shifted predictions toward walking-class labels even without actual walking motion.

Practical Improvements

1. **Add a confidence threshold with a rejection class.** When `best_score` falls below a threshold like 0.75, output "Unknown" instead of the argmax. This would suppress the spurious "Climbing stairs" prediction in Window 2 and prevent low-confidence guesses from being treated as valid outputs.
2. **Collect orientation-aware calibration data.** Since walking and running are frequently confused, fine-tuning or re-normalizing using data collected from the Arduino in known orientations and motion conditions would help the model distinguish these classes more reliably. At minimum, recomputing the normalization constants from board-collected data would reduce the scaling mismatch between training and deployment.

Question 4: Deployment Behavior on the Arduino

Yes, unexpected prediction behavior was observed, though with some physically meaningful patterns. When the board was flat on the surface, the model generally predicted "Lying down," which is correct. When held perpendicular, it shifted toward "Walking," and motion produced "Running" type predictions. However, "Walking" and "Running" were frequently confused with each other, which was the most consistent failure mode.

This confusion is expected given the nature of the training data. In the mHealth dataset, walking and running produce similar periodic accelerometer signatures along the same body axes, differing mainly in magnitude and frequency. On the Arduino, where the exact axis alignment relative to gravity varies with how the board is held, these two classes become even harder to distinguish since the directional component of the signal is not consistent with what was seen during training. The model learned to separate walking from running using subtle differences in body-worn sensor data that do not translate cleanly to a handheld or surface-mounted board.

More broadly, the three base models use different normalization schemes and often produce conflicting high-confidence outputs for the same window, as seen in Window 2 where the raw model was nearly certain of "Running" (0.957) while the other two models disagreed entirely. The meta-classifier was trained to handle disagreements that occurred within the mHealth dataset distribution. When the disagreement pattern on the live board does not match that distribution, the meta-classifier's arbitration becomes unreliable, leading to predictions like "Climbing stairs" on a completely still board.